DOCUMENTATION COMPLÈTE

Professional Profile Explorer

Plateforme Streamlit — cinq modèles intégrés

Table des matières

1. Présentation
2. Principes d'intégration
3. Architecture générale
4. Organisation du dossier
5. Installation sur une autre machine
6. Bibliothèques
7. Chargement des modèles
8. Profil saisi
9. Chaîne de prédiction
10. Module 1 — Ahmed Bejaoui
11. Module 2 — Ahmed Guesmi
12. Module 3 — Nour Trabelsi
13. Module 4 — Arij Yousfi
14. Module 5 — Seifeddine Alaimi
15. Origine des fichiers de modèles
16. Reproduction des modèles
17. Absence d'endpoints personnalisés
18. Portabilité
19. Utilisation sur le réseau local
20. Tests réalisés
21. Dépannage
22. Limites
23. Confidentialité
24. Maintenance
25. Résumé du démarrage
26. Flux d'exécution exhaustif
27. Exécution détaillée des cinq modèles
28. Standardisation des résultats
29. Assemblage et affichage des cartes
30. Cycle de réexécution Streamlit
31. Graphe des appels
32. Exemple complet tracé
33. Gestion des erreurs
34. Performance et mémoire
35. Pseudocode complet
36. Liste de vérification technique
37. Ce que la plateforme ne fait pas

1. Présentation

Professional Profile Explorer est une application Streamlit qui réunit les cinq modules de machine learning du groupe dans une seule carte profil.

L'utilisateur saisit un profil socio-professionnel. La plateforme prépare ces informations selon les règles propres à chaque notebook, exécute les cinq modèles entraînés et affiche leurs résultats dans cinq cartes.

La plateforme repose sur le jeu de données historique **Adult / Census Income de 1994**. Elle constitue un prototype pédagogique d'exploration. Elle ne doit pas être utilisée comme outil de recrutement, de décision administrative ou de garantie sur la situation réelle d'une personne.

2. Principes d'intégration

L'intégration respecte les règles suivantes :

- aucun notebook du dossier `final notebooks` n'est modifié ;
- les hyperparamètres, variables et règles de feature engineering des modèles restent ceux des notebooks ;
- les modèles sont chargés depuis des fichiers portables `.joblib` ;
- les cinq prédictions sont exécutées dans le même processus Python ;
- aucun endpoint API personnalisé n'est nécessaire ;
- le revenu estimé par le module 1 est transmis automatiquement aux modules qui utilisent la variable `income` ;
- les modèles sont chargés une seule fois grâce au cache Streamlit.

3. Architecture générale

```
Navigateur
    |
    | interface Streamlit
    v
streamlit_app.py
    |
    +-- collecte les champs du profil
    +-- crée une ligne pandas DataFrame
    +-- exécute Ahmed Bejaoui en premier
    +-- transmet le revenu estimé aux autres modules
    |
    +--> ahmed_bejaoui.joblib
    +--> ahmed_guesmi.joblib
    +--> nour_trabelsi.joblib
    +--> arij_yousfi.joblib
    +--> seifeddine_alaimi.joblib
    |
    v
Cinq cartes de résultats
```

Tout se déroule localement dans le processus Streamlit. Les fonctions Python appellent directement `predict()` ou `predict_proba()` sur les modèles.

4. Organisation du dossier

```
platform/
├── .streamlit/
│   └── config.toml
├── models/
│   ├── ahmed_bejaoui.joblib
│   ├── ahmed_guesmi.joblib
│   ├── nour_trabelsi.joblib
│   ├── arij_yousfi.joblib
│   └── seifeddine_alaimi.joblib
├── DOCUMENTATION.md
├── README.md
├── requirements.txt
├── run_platform.py
├── streamlit_app.py
└── build_models.py
```

Les anciens fichiers `app.py`, `engine.py`, `templates/` et `static/` appartiennent à la précédente version Flask. Ils ne sont pas appelés par la version Streamlit.

Rôle des fichiers actifs

`streamlit_app.py`

Point d'entrée fonctionnel de la plateforme :

- définit les valeurs proposées dans les listes ;
- charge les cinq modèles ;
- affiche le formulaire ;
- prépare les variables propres à chaque modèle ;
- exécute les prédictions ;
- affiche les résultats.

`run_platform.py`

Lance Streamlit avec le même interpréteur Python que celui utilisé pour exécuter la commande :

```
python run_platform.py
```

Cette méthode évite de dépendre d'une commande `streamlit` globale.

`models/`

Contient les cinq modèles sérialisés nécessaires à l'inférence. Ce dossier doit toujours rester à côté de `streamlit_app.py`.

`requirements.txt`

Fixe les versions des bibliothèques utilisées lors de la création et du chargement des modèles.

`.streamlit/config.toml`

Définit le thème visuel de l'application et le fonctionnement du serveur Streamlit.

`build_models.py`

Script de reproduction des fichiers `.joblib`. Il n'est pas exécuté pendant l'utilisation normale de la plateforme.

5. Installation sur une autre machine

5.1 Prérequis

- Python 3.11 ou plus récent ;
- une connexion Internet uniquement pour installer les bibliothèques ;
- le dossier `platform` complet, y compris son sous-dossier `models`.

Les notebooks et `adult.data` ne sont pas nécessaires pour lancer l'application déjà préparée.

5.2 Installation sur macOS ou Linux

Ouvrir un terminal dans le dossier du projet :

```
cd platform
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python run_platform.py
```

5.3 Installation sur Windows PowerShell

```
cd platform
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python run_platform.py
```

Si PowerShell bloque l'activation de l'environnement :

```
Set-ExecutionPolicy -Scope Process Bypass
.\.venv\Scripts\Activate.ps1
```

5.4 Ouverture

Après le démarrage, ouvrir :

```
http://localhost:8501
```

Pour arrêter l'application, revenir au terminal et utiliser `Ctrl+C`.

6. Bibliothèques

La plateforme utilise :

- `streamlit` pour l'interface ;
- `pandas` pour construire les profils et leurs variables ;
- `numpy` pour les calculs numériques ;
- `scikit-learn` pour les pipelines, transformations et K-Means ;
- `xgboost` pour les modèles supervisés ;
- `joblib` pour charger les modèles ;
- `imbalanced-learn` uniquement si les modèles d'Ahmed Guesmi sont reproduits avec `build_models.py`.

Les versions exactes se trouvent dans `requirements.txt`. Il est recommandé de les conserver, car les objets `joblib` contenant des modèles scikit-learn/XGBoost ne sont pas garantis compatibles entre toutes les versions.

7. Chargement des modèles

La fonction `load_models()` associe chaque module à son fichier :

- `income` → `ahmed_bejaoui.joblib` ;
- `occupation` → `ahmed_guesmi.joblib` ;
- `rhythm` → `nour_trabelsi.joblib` ;
- `education` → `arij_yousfi.joblib` ;
- `cluster` → `seifeddine_alaimi.joblib`.

Le décorateur `@st.cache_resource` garde les objets chargés en mémoire. Un changement de champ dans l'interface relance le script Streamlit, mais ne relit pas les modèles sur le disque à chaque fois.

Si un fichier est absent, la carte concernée affiche :

> `Modèle absent du dossier platform/models.`

8. Profil saisi

La partie gauche de l'interface recueille :

- l'âge, de 17 à 90 ans ;
- le niveau d'études Adult, de `Preschool` à `Doctorate` ;
- les heures de travail hebdomadaires, de 1 à 99 ;

- le secteur d'emploi (`workclass`) ;
- le métier déclaré (`occupation`) ;
- le lien familial (`relationship`) ;
- la situation familiale (`marital-status`) ;
- le gain en capital ;
- la perte en capital ;
- le pays ;
- le sexe et l'origine, dans une section séparée.

Le niveau d'études affiché est converti en `education-num` de 1 à 16.

Le sexe et l'origine sont transmis uniquement au module d'Arij, conformément à son notebook. Les quatre autres modules ne les utilisent pas.

9. Chaîne de prédiction

Étape 1 — Création du profil commun

Les champs de l'interface sont rassemblés dans un DataFrame pandas contenant une seule ligne.

Étape 2 — Revenu estimé

Le modèle d'Ahmed Bejaoui est exécuté en premier. Il produit :

- la probabilité de la classe `>50K` ;
- la classe `<=50K` ou `>50K`, selon son seuil de décision ;
- un libellé lisible dans l'interface.

Étape 3 — Propagation de `income`

La classe de revenu prédite est ajoutée au profil sous la forme :

```
<=50K
```

ou :

```
>50K
```

Cette valeur est ensuite utilisée par les modules d'Ahmed Guesmi, Nour, Arij et Seifeddine lorsque leur modèle attend `income`.

Étape 4 — Exécution des quatre autres modules

Chaque adaptateur applique le feature engineering attendu par son modèle, respecte l'ordre des colonnes d'entraînement, puis exécute la prédiction.

Étape 5 — Affichage

`render_card()` affiche :

- le nom du membre ;
- le titre du module ;
- la classe ou le persona retenu ;
- les probabilités disponibles ;
- une explication courte.

10. Module 1 — Ahmed Bejaoui

Objectif

Classification binaire du revenu :

- `<=50K` ;
- `>50K`.

Modèle

Pipeline XGBoost sauvegardé dans :

```
models/ahmed_bejaoui.joblib
```

Variables utilisées

Variables originales :

- `relationship` ;
- `education-num` ;
- `capital-gain` ;
- `age` ;
- `capital-loss`.

Variables créées :

- `workload` ;

- `workclass-occupation` ;
- `occupation-group` .

Feature engineering

`workload` contient :

- `part-time` pour moins de 35 heures ;
- `full-time` pour 35 à 40 heures ;
- `overtime` au-dessus de 40 heures.

`workclass-occupation` concatène le secteur et le métier.

`occupation-group` regroupe les métiers selon le dictionnaire sauvegardé avec le modèle.

Prédiction

La plateforme appelle :

```
model["pipeline"].predict_proba(...)
```

La barre représente toujours la probabilité de `>50K` .

Le seuil de décision sauvegardé est **0,375** :

- probabilité inférieure à 0,375 → `<=50K` ;
- probabilité supérieure ou égale à 0,375 → `>50K` .

Le résultat de ce module alimente les quatre modules suivants.

11. Module 2 — Ahmed Guesmi

Objectif

Classification multiclasse d'une famille de métiers :

- Bureau, vente & protection ;
- Cadres & professions intellectuelles ;
- Manuel & technique ;
- Services & manutention.

Le module fournit aussi un métier indicatif appartenant à la famille prédite.

Modèles

Le fichier `ahmed_guesmi.joblib` contient :

- un XGBoost pour la famille de métiers ;
- un XGBoost pour les 14 métiers ;
- le dictionnaire métier → famille ;
- les encodeurs de classes ;
- le codage de `workclass` ;
- les seuils de confiance.

Variables utilisées

- `education-num` ;
- `age` ;
- `hours-per-week` ;
- revenu estimé ;
- `workclass` ;
- `education-level` créé ;
- `age-group` créé ;
- `workload` créé.

Le métier saisi n'est pas une entrée du modèle : il correspond à la cible historique que le module cherche à estimer.

Gestion de l'incertitude

- confiance inférieure à 0,53 → `Profil trop incertain` ;
- confiance de 0,53 à moins de 0,68 → fiabilité modérée ;
- confiance supérieure ou égale à 0,68 → fiabilité élevée.

Le métier indicatif est choisi parmi les métiers appartenant à la famille prédite.

12. Module 3 — Nour Trabelsi

Objectif

Classification multiclasse du rythme hebdomadaire probable :

- `Partiel (<35 h)` ;
- `Standard (35-40 h)` ;
- `Plus de 40 h`.

Modèle

XGBoost avec préprocesseur sauvegardé dans :

```
models/nour_trabelsi.joblib
```

Variables utilisées

- `age` ;
- `education-num` ;
- `occupation` ;
- `workclass` ;
- revenu estimé ;
- `relationship` ;
- `age-group` créé ;
- `education-level` créé ;
- `employment-sector` créé.

Point important

`hours-per-week` définit la cible d'entraînement et n'est donc pas utilisé comme entrée de ce modèle. La carte de Nour n'utilise pas la valeur d'heures saisie par l'utilisateur.

Le préprocesseur :

- remplace les valeurs numériques manquantes par la médiane ;
- standardise les variables numériques ;
- remplace les catégories manquantes par la modalité la plus fréquente ;
- applique un One-Hot Encoding ;
- conserve l'ordre exact des variables appris par XGBoost.

La plateforme affiche les trois probabilités dans l'ordre décroissant.

13. Module 4 — Arij Yousfi

Objectif

Classification binaire du niveau d'études historiquement associé au profil :

- `Sous la licence` ;
- `Licence ou plus`.

La cible `Licence ou plus` correspond à `education-num >= 13`.

Modèle

XGBoost sauvegardé dans :

```
models/arij_yousfi.joblib
```

Préparation

La plateforme reproduit les transformations du notebook :

- revenu en 0/1 ;
- sexe en 0/1 ;
- tranche d'âge ;
- tranche de charge de travail ;
- secteur d'emploi ;
- présence d'un capital ;
- One-Hot Encoding des variables nominales ;
- réalignement sur les 39 colonnes retenues pendant l'entraînement.

Les colonnes absentes du profil deviennent 0 grâce à :

```
reindex(columns=model["columns"], fill_value=0)
```

Point important

Le niveau d'études sélectionné dans le formulaire n'est pas transmis à ce module, car il représente la cible à prédire. Il reste utilisé par les autres modules.

Le seuil de décision est 0,50.

Le sexe et l'origine sont utilisés par ce module parce qu'ils figurent dans le modèle final de son notebook. L'interface les place dans une section séparée pour rendre cette utilisation visible.

14. Module 5 — Seifeddine Alaimi

Objectif

Segmentation non supervisée des profils professionnels avec K-Means.

Modèle

Le fichier `seifeddine_alaimi.joblib` contient :

- le préprocesseur ;
- le K-Means ;
- la correspondance entre labels techniques et groupes affichés ;
- les noms des personas ;
- les bornes des tranches d'âge et d'heures ;
- l'ordre des colonnes.

Variables utilisées

Numériques :

- `age` ;
- `education-num` ;
- `hours-per-week` ;
- `net_capital_log` ;
- `work_life_ratio`.

Ordinales :

- `age_group` ;
- `workload`.

Catégorielles :

- `occupation` ;
- `workclass` ;
- revenu estimé.

Variables créées

```
net_capital = capital-gain - capital-loss
```

`net_capital_log` applique une transformation logarithmique signée.

```
work_life_ratio = hours-per-week / age
```

Personas

Les quatre groupes sont :

- Actifs confirmés à temps plein ;
- Jeunes actifs en début de carrière ;
- Professionnels suractifs ;
- Hauts revenus à fort capital.

Le sexe et l'origine ne participent pas à la formation des groupes.

15. Origine des fichiers de modèles

Modèles déjà exportés par leurs notebooks

Les modèles suivants existaient déjà sous forme `.joblib` :

- Ahmed Bejaoui ;
- Nour Trabelsi.

Ils ont été copiés dans `platform/models` sous des noms uniformes.

Notebooks sans artefact d'inférence complet

Les notebooks d'Ahmed Guesmi, Arij Yousfi et Seifeddine Alaimi ne laissaient pas tous un fichier unique directement utilisable par la plateforme.

`build_models.py` reprend leurs règles finales, leurs hyperparamètres, leurs variables et leurs graines aléatoires afin de produire les fichiers portables. Cette opération ne modifie pas les notebooks.

16. Reproduction des modèles

Cette section n'est utile que si un fichier du dossier `models` est perdu ou si le groupe décide volontairement de régénérer les artefacts.

Prérequis supplémentaires à la racine du projet :

- `adult/adult.data` ;
- `member1_xgboost_best.joblib` ;
- `member3_rhythm_xgboost.joblib`.

Commande :

```
cd platform
python build_models.py
```

Cette commande remplace les cinq fichiers dans `platform/models`. Elle ne doit pas être exécutée pour simplement lancer l'application.

17. Absence d'endpoints personnalisés

La version Streamlit ne définit pas de routes telles que :

```
/api/profile
/api/status
/api/choices
```

Ces routes appartenaient à l'ancienne version Flask.

Streamlit gère lui-même la communication HTTP/WebSocket avec le navigateur. Le code du projet ne définit aucune API REST : les modèles sont appelés directement par des fonctions Python dans `streamlit_app.py`.

18. Portabilité

Pour transférer la plateforme à un membre du groupe :

1. compresser ou copier le dossier `platform` complet ;
2. vérifier que les cinq fichiers existent dans `platform/models` ;
3. installer les dépendances depuis `requirements.txt` ;
4. exécuter `python run_platform.py`.

Il ne faut pas copier seulement `streamlit_app.py`, car l'application a besoin du dossier `models`, du fichier de configuration et des dépendances.

19. Utilisation sur le réseau local

Pour rendre temporairement l'application accessible aux personnes connectées au même réseau Wi-Fi :

```
python -m streamlit run streamlit_app.py --server.address 0.0.0.0
```

Streamlit affiche alors une `Network URL`. Les autres membres peuvent ouvrir cette adresse si le pare-feu de la machine l'autorise.

Cette méthode ne doit pas être utilisée pour exposer directement l'application sur Internet sans authentification, HTTPS et configuration serveur adaptée.

20. Tests réalisés

L'intégration a été testée de deux façons :

- appel direct des cinq fonctions de prédiction ;
- vérification visuelle dans un navigateur sur `localhost:8501`.

Profil courant

- 35 ans ;
- HS-grad ;
- 40 heures ;
- Private ;
- Craft-repair ;
- capital nul.

Résultats observés :

- revenu : 50 000 $ ou moins ;
- famille de métiers : profil trop incertain ;
- rythme : Standard ;
- éducation associée : Sous la licence ;
- persona : Jeunes actifs en début de carrière.

Profil à capital extrême

- 22 ans ;
- Bachelors ;
- 40 heures ;
- Sales ;
- capital-gain de 99 999.

Résultats observés :

- revenu : Plus de 50 000 $;
- famille : Cadres & professions intellectuelles, avec fiabilité modérée ;
- rythme : Standard ;
- éducation associée : Sous la licence ;
- persona : Hauts revenus à fort capital.

Profil cadre

- 45 ans ;

- Masters ;
- 50 heures ;
- Exec-managerial.

Résultats observés :

- revenu : Plus de 50 000 $;
- famille : Cadres & professions intellectuelles ;
- rythme : Plus de 40 heures ;
- éducation associée : Licence ou plus ;
- persona : Professionnels suractifs.

Ces scénarios vérifient le chargement des cinq artefacts, le feature engineering et la propagation du revenu.

21. Dépannage

`ModuleNotFoundError`

Cause probable : les dépendances ne sont pas installées dans l'interpréteur Python actif.

Solution :

```
python -m pip install -r requirements.txt
```

Vérifier également que l'environnement virtuel est activé.

`Modèle absent du dossier platform/models`

Vérifier la présence des cinq fichiers `.joblib`. Le dossier `models` doit être au même niveau que `streamlit_app.py`.

Erreur de compatibilité Joblib, scikit-learn ou XGBoost

Créer un nouvel environnement virtuel puis installer exactement les versions de `requirements.txt`.

Le port 8501 est déjà utilisé

Lancer sur un autre port :

```
python -m streamlit run streamlit_app.py --server.port 8502
```

Puis ouvrir :

```
http://localhost:8502
```

La page ne se met pas à jour

Arrêter le serveur avec `Ctrl+C`, le relancer, puis actualiser le navigateur. Le cache peut aussi être vidé depuis le menu Streamlit.

Le navigateur ne s'ouvre pas automatiquement

Ouvrir manuellement :

```
http://localhost:8501
```

22. Limites

- Les données Adult datent de 1994.
- Les résultats décrivent des associations historiques, pas des causalités.
- La première prédiction de revenu influence quatre cartes suivantes ; une erreur du premier module peut donc se propager.
- Une probabilité n'est pas une certitude individuelle.
- Le module d'Ahmed Guesmi peut s'abstenir lorsque la confiance est trop faible.
- Un cluster est un groupe de proximité statistique, pas une vérité sur une personne.
- Les catégories proposées restent limitées aux modalités apprises dans Adult.
- Les objets `.joblib` doivent être considérés comme du code Python : charger uniquement les fichiers fournis par le groupe.
- Le modèle d'Arij utilise des variables sensibles présentes dans son notebook ; cette utilisation doit être discutée dans l'analyse d'équité.

23. Confidentialité

La plateforme ne contient pas de base de données et n'enregistre pas les profils dans un fichier. Les valeurs sont conservées dans la session Streamlit pendant l'utilisation.

Pour un usage autre que la démonstration locale, il faudrait ajouter :

- une politique de confidentialité ;
- une authentification ;
- HTTPS ;
- une gestion explicite des journaux ;

- des contrôles d'accès ;
- un hébergement sécurisé.

24. Maintenance

Remplacer un modèle

Un fichier peut être remplacé seulement si son nouveau contenu respecte le contrat attendu par sa fonction dans `streamlit_app.py`.

Par exemple, le modèle de Nour doit conserver :

- `model` ;
- `preprocessor` ;
- `features` ;
- `feature_names` ;
- `labels`.

Une modification de clés, de noms de colonnes ou d'ordre des variables exige une adaptation du code d'inférence.

Ajouter un module

Pour ajouter un sixième module :

1. exporter son modèle en `.joblib` ;
2. l'ajouter dans `load_models()` ;
3. créer une fonction d'adaptation et de prédiction ;
4. ajouter une carte dans la liste `cards` ;
5. tester au moins un profil courant et un profil extrême.

25. Résumé du démarrage

Sur toute machine disposant de Python :

```
cd platform
python -m pip install -r requirements.txt
python run_platform.py
```

Adresse :

```
http://localhost:8501
```

La plateforme fonctionne ensuite sans réexécuter les notebooks et sans réentraîner les modèles.

26. Flux d'exécution exhaustif

Cette section décrit chronologiquement tout ce que fait la plateforme, depuis la commande de lancement jusqu'à l'affichage des cinq résultats.

Phase A — Lancement du processus

Étape 1 — Activation de l'environnement

L'utilisateur active l'environnement Python dans lequel les dépendances de `requirements.txt` sont installées.

Cette étape garantit que Streamlit, pandas, scikit-learn, XGBoost et Joblib proviennent du même environnement.

Étape 2 — Exécution du lanceur

La commande recommandée est :

```
python run_platform.py
```

`run_platform.py` récupère :

```
sys.executable
```

Cette valeur est le chemin de l'interpréteur Python actuellement utilisé. Le lanceur exécute ensuite l'équivalent de :

```
python -m streamlit run /chemin/absolu/vers/streamlit_app.py
```

L'utilisation du chemin absolu évite de dépendre du nom du dossier parent ou du répertoire personnel de l'utilisateur.

Étape 3 — Démarrage du serveur Streamlit

Streamlit démarre un serveur local, normalement sur le port 8501.

Il fournit :

- une adresse locale, généralement `http://localhost:8501` ;
- éventuellement une adresse réseau local ;
- une connexion gérée par Streamlit entre le navigateur et Python.

La plateforme ne démarre aucun serveur Flask et ne crée aucune API REST.

Étape 4 — Lecture de la configuration

Quand l'application est lancée depuis `platform`, Streamlit lit :

```
.streamlit/config.toml
```

Ce fichier définit notamment :

- le thème clair ;
- la couleur principale ;
- la couleur de fond ;
- la couleur des panneaux ;
- la couleur du texte ;
- le mode serveur sans interface système supplémentaire.

Phase B — Chargement du script Python

Étape 5 — Importation des bibliothèques

Python importe :

- `Path` pour les chemins ;
- `joblib` pour les artefacts ;
- `numpy` pour les calculs ;
- `pandas` pour les DataFrames ;
- `streamlit` pour l'interface.

À ce stade, aucun modèle n'est encore exécuté.

Étape 6 — Construction du chemin des modèles

La constante `MODELS` est construite à partir de l'emplacement réel de `streamlit_app.py` :

```
MODELS = Path(__file__).resolve().parent / "models"
```

Si le projet est copié sur une autre machine, ce chemin devient automatiquement le dossier `models` de cette nouvelle copie.

Il n'existe donc aucun chemin absolu dépendant de la machine d'Ahmed.

Étape 7 — Déclaration des modalités

Python construit les listes utilisées par les widgets :

- niveaux d'études ;
- secteurs professionnels ;
- métiers ;
- liens familiaux ;
- situations familiales ;
- pays ;
- sexes ;
- origines.

Ces valeurs reprennent les modalités du jeu Adult que les modèles ont vues pendant leur entraînement.

Étape 8 — Déclaration des profils de démonstration

Deux dictionnaires sont créés en mémoire :

- `Parcours courant` ;
- `Capital extrême`.

Ils ne déclenchent aucune prédiction tant que l'interface n'appelle pas `apply_profile()`.

Phase C — Construction initiale de la page

Étape 9 — Appel de `main()`

À la fin du fichier :

```
if __name__ == "__main__":
    main()
```

Streamlit exécute alors `main()`.

Étape 10 — Configuration de la page

`st.set_page_config()` définit :

- le titre de l'onglet ;
- l'icône ;
- la disposition large.

Étape 11 — Injection du style

`st.markdown(..., unsafe_allow_html=True)` ajoute :

- le bandeau principal ;
- les couleurs ;
- les espacements ;
- la taille du résultat ;
- la bordure des cartes.

Ce CSS affecte uniquement la présentation. Il ne change aucune prédiction.

Étape 12 — Initialisation de la session

Streamlit maintient un `session_state` propre à chaque session navigateur.

Si la clé `age` n'existe pas encore, `apply_profile()` copie le profil `Parcours courant` dans cet état.

Cette opération fournit des valeurs valides à tous les widgets lors du premier affichage.

Phase D — Chargement des cinq artefacts

Étape 13 — Appel de `load_models()`

`main()` appelle :

```
models = load_models()
```

La fonction associe cinq clés internes à cinq fichiers :

```
income      → ahmed_bejaoui.joblib
occupation  → ahmed_guesmi.joblib
rhythm      → nour_trabelsi.joblib
education   → arij_yousfi.joblib
cluster     → seifeddine_alaimi.joblib
```

Étape 14 — Vérification de chaque fichier

Pour chaque nom, la plateforme construit son chemin puis vérifie :

```
path.exists()
```

- fichier présent → `joblib.load(path)` ;
- fichier absent → valeur `None`.

L'absence d'un modèle n'empêche donc pas automatiquement le chargement de la page.

Étape 15 — Désérialisation Joblib

`joblib.load()` reconstruit en mémoire :

- les estimateurs XGBoost ;
- les pipelines scikit-learn ;
- les préprocesseurs ;
- les listes de variables ;
- les noms de classes ;
- les seuils ;
- les dictionnaires nécessaires à l'inférence.

Un fichier `.joblib` doit uniquement être chargé s'il provient d'une source de confiance, car il peut contenir des objets Python exécutables.

Étape 16 — Mise en cache

`load_models()` utilise :

```
@st.cache_resource
```

Au premier passage, les modèles sont lus depuis le disque.

Lors des interactions suivantes, Streamlit réutilise les objets déjà en mémoire. Les modèles ne sont donc pas rechargés à chaque mouvement de slider.

Phase E — Construction du formulaire

Étape 17 — Création des deux colonnes

La page est séparée en :

- colonne gauche : formulaire ;
- colonne droite : résultats.

Étape 18 — Boutons de démonstration

Le clic sur un bouton appelle `apply_profile()` avec le dictionnaire choisi.

La fonction copie chaque paire clé/valeur dans `st.session_state`.

`st.rerun()` relance immédiatement le script afin que tous les widgets affichent les nouvelles valeurs.

Étape 19 — Création des widgets

Streamlit crée :

- deux sliders ;
- plusieurs listes déroulantes ;
- deux entrées numériques ;
- un panneau repliable pour le sexe et l'origine.

Chaque widget possède une clé dans `session_state`.

Étape 20 — Récupération des valeurs

Les widgets renvoient directement leurs valeurs Python :

- `age` et `hours` sont des entiers ;
- `gain` et `loss` sont numériques ;
- les sélections catégorielles sont des chaînes.

Étape 21 — Conversion du diplôme

Le libellé choisi, par exemple :

```
Bachelors
```

est converti en code Adult :

```
13
```

grâce à :

```
dict(EDUCATION)[education_name]
```

Phase F — Création du profil commun

Étape 22 — Construction du DataFrame

Les valeurs sont réunies dans un DataFrame pandas d'une ligne appelé `raw`.

Ses colonnes sont :

```
age
education-num
hours-per-week
workclass
occupation
relationship
marital-status
capital-gain
capital-loss
native-country
```

Le sexe et l'origine ne sont pas ajoutés à ce DataFrame commun. Ils sont transmis séparément au seul module d'Arij.

Étape 23 — Protection contre les mutations

Chaque fonction de modèle commence par copier le profil avec :

```
raw.copy()
```

Les variables créées pour un module ne modifient donc pas le DataFrame utilisé par les autres modules.

27. Exécution détaillée des cinq modèles

Module 1 — Revenu

Étape 24 — Création de `workload`

Les heures sont transformées en :

```
moins de 35  → part-time
35 à 40      → full-time
plus de 40   → overtime
```

Étape 25 — Création de `workclass-occupation`

La plateforme concatène le secteur et le métier :

```
Private + Craft-repair → Private__Craft-repair
```

Étape 26 — Création de `occupation-group`

Le métier est converti vers son groupe au moyen du dictionnaire sauvegardé avec le modèle.

Une modalité non reconnue reçoit `unknown`.

Étape 27 — Sélection et ordre des variables

La plateforme utilise exactement :

```
featured[model["features"]]
```

Cette sélection est importante : un pipeline scikit-learn attend les mêmes noms et le même ordre logique qu'à l'entraînement.

Étape 28 — Calcul de la probabilité

Le pipeline complet reçoit les variables et exécute son prétraitement puis XGBoost :

```
model["pipeline"].predict_proba(...)
```

`[0, 1]` sélectionne :

- la première personne, car le DataFrame contient une ligne ;
- la probabilité de la classe positive `>50K`.

Étape 29 — Application du seuil

La probabilité est comparée au seuil sauvegardé :

```
0,375
```

Le résultat devient :

- `<=50K` si la probabilité est inférieure ;
- `>50K` si elle est supérieure ou égale.

Étape 30 — Création du résultat revenu

La fonction retourne un dictionnaire contenant :

- `title` : texte destiné à la carte ;
- `label` : valeur technique `<=50K` ou `>50K` ;
- `score` : probabilité numérique ;
- `bars` : valeurs à afficher ;

- `note` : explication du seuil.

Propagation du revenu

Étape 31 — Extraction de `income_label`

`main()` récupère :

```
income_label = income["label"]
```

Cette chaîne devient une entrée des modules suivants.

Si le fichier du revenu est absent, la valeur de secours actuelle est :

```
<=50K
```

La carte d'Ahmed indique alors que son modèle est absent, tandis que les autres modules peuvent encore être exécutés avec cette valeur de secours.

Module 2 — Famille de métiers

Étape 32 — Ajout du revenu

Le revenu estimé est ajouté à une copie du profil.

Étape 33 — Construction de la matrice encodée

La fonction crée :

- `education-num` ;
- `age` ;
- `hours-per-week` ;
- `income` codé en 0/1 ;
- `workclass` codé par le dictionnaire du modèle ;
- `education-level` ;
- `age-group` ;
- `workload`.

Une valeur `workclass` inconnue reçoit le code `-1`.

Le métier sélectionné dans le formulaire n'est pas copié dans cette matrice : il représente la cible que ce module essaie de prédire.

Étape 34 — Prédiction de la famille

Le premier XGBoost calcule quatre probabilités avec :

```
family_model.predict_proba(encoded)
```

`np.argmax()` identifie la classe la plus probable.

Les résultats sont ensuite triés du plus probable au moins probable.

Étape 35 — Décision d'abstention

La confiance maximale est comparée au seuil modéré :

```
0,53
```

Sous ce seuil, le module retourne :

```
Profil trop incertain
```

Il affiche quand même les quatre probabilités.

Étape 36 — Prédiction du métier indicatif

Si la confiance est suffisante, le second XGBoost calcule les probabilités des 14 métiers.

La plateforme :

1. conserve uniquement les métiers appartenant à la famille gagnante ;
2. sélectionne le métier ayant la probabilité maximale parmi eux.

Étape 37 — Niveau de fiabilité

- confiance de 0,53 à moins de 0,68 → modérée ;
- confiance d'au moins 0,68 → élevée.

Module 3 — Rythme de travail

Étape 38 — Ajout du revenu

Le revenu estimé est ajouté au profil.

Étape 39 — Création des groupes

La fonction crée :

- `age-group` avec quatre tranches ;
- `education-level` avec cinq tranches ;
- `employment-sector`.

Le secteur devient :

- `private` ;
- `public` ;
- `self-employed` ;
- `other` ;
- `unknown`.

Étape 40 — Sélection des variables

La plateforme sélectionne les colonnes sauvegardées dans :

```
model["features"]
```

`hours-per-week` n'appartient pas à cette liste. Même si la valeur existe dans `raw`, elle n'entre donc pas dans le modèle de Nour.

Étape 41 — Prétraitement

Le préprocesseur sauvegardé :

- impute les valeurs manquantes ;
- standardise les nombres ;
- encode les catégories ;
- produit la matrice attendue par XGBoost.

Étape 42 — Restauration des noms de colonnes

Si la matrice est sparse, elle est transformée en tableau dense.

Un DataFrame est reconstruit avec :

```
model["feature_names"]
```

Cette étape respecte les noms nettoyés utilisés pendant l'entraînement de XGBoost.

Étape 43 — Probabilités de rythme

XGBoost retourne trois probabilités. `np.argsort(-probabilities)` les classe par ordre décroissant.

La première devient le titre de la carte.

Module 4 — Niveau d'études

Étape 44 — Construction des variables numériques

La fonction récupère :

- âge ;
- heures ;
- gain et perte en capital ;
- revenu estimé.

Elle ajoute :

- sexe codé ;
- `has-capital` ;
- tranche d'âge ;
- tranche de charge de travail.

`education-num` n'est pas repris, car il définit la cible de ce module.

Étape 45 — Construction des catégories

Les variables catégorielles sont :

- métier ;
- secteur ;
- situation familiale ;
- lien familial ;
- origine ;
- pays ;
- secteur d'emploi regroupé.

Étape 46 — One-Hot Encoding manuel

Pour chaque catégorie, la plateforme crée un nom de colonne de la forme :

```
occupation_Craft-repair
workclass_Private
race_White
```

La valeur de la modalité active est 1.

Étape 47 — Alignement exact

Le DataFrame est réaligné sur les 39 colonnes sauvegardées :

```
reindex(columns=model["columns"], fill_value=0)
```

Cette opération :

- place les colonnes dans l'ordre attendu ;
- ajoute les colonnes manquantes avec 0 ;
- ignore les colonnes qui n'ont pas été sélectionnées à l'entraînement.

Étape 48 — Prédiction

XGBoost calcule la probabilité de `Licence ou plus`.

Le seuil 0,50 produit :

- `Sous la licence` ;
- `Licence ou plus`.

Module 5 — Segmentation

Étape 49 — Ajout du revenu

Le revenu estimé est ajouté au profil comme variable catégorielle.

Étape 50 — Capital net

La plateforme calcule :

```
capital-gain - capital-loss
```

Étape 51 — Transformation logarithmique signée

La formule utilisée est :

```
sign(capital net) × log(1 + |capital net|)
```

Elle réduit l'influence des valeurs de capital très élevées sans supprimer le signe des pertes.

Étape 52 — Variables de rythme et d'âge

Les tranches sont créées avec les bornes et les libellés sauvegardés dans l'artefact du modèle.

Étape 53 — Ratio travail/âge

La plateforme calcule :

```
hours-per-week / age
```

Étape 54 — Prétraitement

Le pipeline de Seifeddine :

- standardise les variables numériques ;
- applique un encodage ordinal aux tranches ;
- applique un One-Hot Encoding aux catégories.

Étape 55 — Affectation K-Means

`kmeans.predict()` choisit le centroïde le plus proche et retourne un label technique.

Étape 56 — Renumérotation et persona

Le dictionnaire `mapping` convertit le label technique vers le numéro stable affiché.

`names` convertit ce numéro vers un persona lisible.

K-Means ne retourne pas une probabilité ; la carte affiche donc uniquement le groupe et son nom.

28. Standardisation des résultats

Les cinq fonctions retournent des dictionnaires conçus pour être compris par une seule fonction d'affichage.

Structure commune

```
{
    "title": "résultat principal",
    "bars": [
        ("nom de la classe", probabilité),
    ],
    "note": "explication"
}
```

Champs supplémentaires du revenu

Le module revenu ajoute :

```
{
    "label": "<=50K ou >50K",
    "score": 0.0 à 1.0
}
```

`label` sert à la communication entre les modules.

`title` sert uniquement à l'affichage utilisateur.

29. Assemblage et affichage des cartes

Étape 57 — Création de la liste `cards`

Python évalue chaque appel dans l'ordre de la liste :

1. résultat revenu déjà calculé ;
2. famille de métiers ;
3. rythme ;
4. niveau d'études ;
5. groupe professionnel.

Chaque entrée contient :

- un numéro ;
- un membre ;
- un titre ;
- un résultat.

Étape 58 — Gestion d'un modèle absent

Si un artefact vaut `None`, le résultat de sa carte vaut aussi `None`.

`render_card()` affiche alors :

```
Modèle absent du dossier platform/models.
```

Étape 59 — Affichage du titre

`render_card()` affiche le numéro, le nom du module et le membre.

Étape 60 — Affichage du résultat principal

Le champ `title` est affiché avec la classe CSS `.result`.

Étape 61 — Affichage des probabilités

Chaque paire de `bars` devient une barre Streamlit.

La valeur est limitée entre 0 et 1 avant l'affichage :

```
min(max(value, 0.0), 1.0)
```

Étape 62 — Affichage de la note

Le champ `note` devient une légende sous la carte.

30. Cycle de réexécution Streamlit

Streamlit adopte un modèle d'exécution différent d'une page HTML classique.

Quand l'utilisateur change un champ :

1. la nouvelle valeur est enregistrée dans la session ;
2. `streamlit_app.py` est réexécuté du début à la fin ;
3. les constantes sont recréées ;
4. `main()` reconstruit la page ;
5. `load_models()` récupère les modèles depuis le cache ;
6. le DataFrame `raw` est reconstruit ;
7. les cinq inférences sont recalculées ;
8. les cartes sont redessinées.

Les modèles ne sont pas réentraînés et ne sont normalement pas relus depuis le disque.

État par navigateur

Les valeurs des widgets appartiennent à la session du navigateur concerné.

Ressources partagées

Les objets retournés par `st.cache_resource` sont conservés au niveau du processus serveur et peuvent être réutilisés entre les réexécutions.

La plateforme les utilise uniquement pour l'inférence.

31. Graphe des appels

```
run_platform.py
└── python -m streamlit run streamlit_app.py
    └── main()
        ├── st.set_page_config()
        ├── CSS + bandeau
        ├── apply_profile() si première session
        ├── load_models()
        │   └── joblib.load() × 5 au premier chargement
        ├── widgets Streamlit
        ├── pandas.DataFrame([profil])
        ├── income_card()
        │   └── pipeline.predict_proba()
        ├── occupation_card()
        │   ├── family_model.predict_proba()
        │   └── occupation_model.predict_proba()
        ├── rhythm_card()
        │   ├── preprocessor.transform()
        │   └── model.predict_proba()
        ├── education_card()
        │   └── model.predict_proba()
        ├── cluster_card()
        │   ├── preprocessor.transform()
        │   └── kmeans.predict()
        └── render_card() × 5
```

32. Exemple complet tracé

Considérons le profil de démonstration courant :

```
age               = 35
education         = HS-grad
education-num     = 9
hours-per-week    = 40
workclass         = Private
occupation        = Craft-repair
relationship      = Husband
marital-status    = Married-civ-spouse
capital-gain      = 0
capital-loss      = 0
native-country    = United-States
sex               = Male
race              = White
```

Passage dans Ahmed Bejaoui

Variables créées :

```
workload             = full-time
workclass-occupation = Private__Craft-repair
occupation-group     = skilled-manual
```

Lors du test d'intégration, le modèle a produit environ 18 % pour `>50K`.

Comme :

```
0,18 < 0,375
```

la classe transmise est :

```
<=50K
```

Passage dans Ahmed Guesmi

`income` devient 0.

Le groupe `Manuel & technique` atteint environ 51 %, mais :

```
0,51 < 0,53
```

La carte affiche donc `Profil trop incertain`.

Passage dans Nour

Le profil reçoit :

```
income             = <=50K
age-group          = adult
education-level    = high-school
employment-sector = private
```

Le modèle a sélectionné `Standard (35-40 h)`.

Passage dans Arij

Le niveau `HS-grad` n'est pas utilisé comme entrée.

Le modèle utilise le reste du profil, dont le revenu estimé, le sexe et l'origine, et sélectionne `Sous la licence`.

Passage dans Seifeddine

Les variables créées comprennent :

```
net_capital_log = 0
work_life_ratio = 40 / 35
```

Le profil est affecté au persona `Jeunes actifs en début de carrière`.

33. Gestion des erreurs

Fichier manquant

Cette situation est gérée explicitement : la carte affiche un message.

Erreur de désérialisation

Si un fichier existe mais ne peut pas être chargé, par exemple à cause d'une version incompatible, `joblib.load()` lève une exception. Streamlit affiche alors son écran d'erreur.

Erreur de colonnes

Si un artefact est remplacé par un modèle ayant d'autres colonnes, la transformation ou la prédiction peut échouer.

Il faut alors mettre à jour simultanément :

- le modèle ;
- les métadonnées de variables ;
- la fonction d'adaptation.

Modalité inconnue

Le comportement dépend du module :

- Ahmed utilise le pipeline sauvegardé ;
- Nour utilise `handle_unknown="ignore"` dans son encodeur ;
- Guesmi remplace un secteur inconnu par `-1` ;
- Arij réaligne les colonnes et remplit les absences par 0 ;
- Seifeddine utilise le préprocesseur sauvegardé.

L'interface limite normalement les entrées aux modalités connues.

Valeur hors limites

Les sliders et entrées numériques empêchent les valeurs en dehors des intervalles définis dans l'interface.

34. Performance et mémoire

Entraînement

Aucun entraînement n'est exécuté par `streamlit_app.py`.

Chargement

Les cinq artefacts représentent environ 11 Mo sur le disque. Ils sont chargés au premier accès puis conservés par le cache.

Inférence

Chaque interaction exécute plusieurs prédictions sur une seule ligne. Cette charge est faible par rapport à l'entraînement.

Réseau

Les modèles n'appellent aucun service externe. Les prédictions sont locales.

La connexion navigateur/serveur reste gérée par Streamlit.

35. Pseudocode complet

```
DÉMARRER Streamlit

CHARGER ou RÉUTILISER les cinq modèles

SI nouvelle session :
    CHARGER le profil courant dans session_state

AFFICHER le formulaire
LIRE les valeurs des widgets
CONVERTIR le diplôme vers education-num
CRÉER raw = DataFrame d'une ligne

SI modèle revenu disponible :
    CRÉER ses variables
    CALCULER P(>50K)
    APPLIQUER seuil 0,375
    PRODUIRE income_label
SINON :
    UTILISER income_label = <=50K comme valeur de secours

SI modèle métier disponible :
    ENCODER le profil avec income_label
    CALCULER les probabilités des familles
    SI confiance < 0,53 :
        S'ABSTENIR
    SINON :
        PRÉDIRE un métier indicatif dans la famille

SI modèle rythme disponible :
    CRÉER les groupes d'âge, d'études et de secteur
    APPLIQUER le préprocesseur
    CALCULER les trois probabilités

SI modèle éducation disponible :
    CRÉER les variables sans utiliser education-num
    ALIGNER sur les 39 colonnes
    CALCULER P(licence ou plus)
    APPLIQUER seuil 0,50

SI modèle clustering disponible :
    CRÉER capital net logarithmique et ratio travail/âge
    APPLIQUER le préprocesseur
    PRÉDIRE le cluster
    CONVERTIR le cluster en persona

AFFICHER les cinq cartes
```

```
ATTENDRE une interaction

À CHAQUE INTERACTION :
    RÉEXÉCUTER le flux en réutilisant les modèles en cache
```

36. Liste de vérification technique

Avant une démonstration sur une nouvelle machine :

1. vérifier `python --version` ;
2. activer l'environnement virtuel ;
3. installer `requirements.txt` ;
4. vérifier les cinq fichiers dans `models/` ;
5. exécuter `python run_platform.py` ;
6. ouvrir `http://localhost:8501` ;
7. tester `Parcours courant` ;
8. tester `Capital extrême` ;
9. vérifier que les cinq cartes répondent ;
10. vérifier que la carte revenu influence les autres modules ;
11. laisser le terminal ouvert pendant la démonstration ;
12. arrêter avec `Ctrl+C` après la démonstration.

37. Ce que la plateforme ne fait pas

La plateforme :

- ne réexécute pas les notebooks ;
- ne réentraîne pas les modèles ;
- ne modifie pas `adult.data` ;
- n'écrit pas les profils dans une base de données ;
- ne crée pas d'endpoint REST personnalisé ;
- n'envoie pas les profils à une API de machine learning ;
- ne remplace pas une validation humaine ;
- ne transforme pas une probabilité en garantie.